

MANUAL TÉCNICO

Aplicación de Rony Alvarez - Proyecto 1- OLC1-

Índice

Analizador léxico y sintáctico.....	4
1. Librerías usadas - jflex.....	4
2. Librerías usadas - cup - (parser).....	5
3. Archivo de compilación.....	6
Interfaz gráfica.....	7
4. Vista del IDE.....	7
5. Controlador de la interfaz.....	7
Clases para materialización de la BD.....	9
6. Clase tabla.....	9
7. Clase actualizacionInfo.....	9
8. Clase campoInfo.....	10
9. Clase dataBase.....	10
10. Clase dataBase.....	10
11. Clase filtroInfo.....	11
12. Clase gestorDB.....	11
13. Clase lecturalInfo.....	17
14. Clase token.....	17
Orden de las carpetas del proyecto.....	18

Introducción

Este manual acerca del software del lenguaje ELI, está hecho para que los desarrolladores interesados en el programa sean capaces de entender la lógica detrás de su funcionamiento, los métodos y funciones usadas en lenguaje de programación Perl.

Objetivo

El objetivo de este manual es ayudar a los desarrolladores, tutores auxiliares, y público en general a comprender el desarrollo de este software, y el pensamiento del desarrollador que lo ha creado.

También está hecho con el objetivo de mostrar al público general que el estudiante de OLC1 ha logrado utilizar los conceptos que fueron planteados antes del práctica, tanto como respetar las reglas de las estructuras que se nos fueron permitidas para usarse.

Dirigidos

Este manual está orientado a todas aquellas personas que estén interesadas en el funcionamiento del software, el desarrollo y lógica detrás de él. Para que en un futuro, por si les es de utilidad puedan utilizar este programa como guía para sus actividades.

Especificación técnica

Requisitos de Hardware

- Computadora de escritorio o portátil.
- 4GB de memoria RAM.
- 256GB de disco duro o más.
- Procesador intel i3, AMD ryzen 3, o superior.
- Resolución gráfica mínima de 1024 x 768 píxeles.

Requisitos de Software

- S.O. Windows 10 o superior.
- Tener instalado jdk 21.
- Tener instalado Jflex.
- Tener instalado Cup.

Especificación técnica

Analizador léxico y sintáctico

1. Librerías usadas - jflex

La librería que usaremos para el análisis léxico será la librería de Jflex. Esta librería necesita un archivo de configuración .jflex para que pueda generar el código necesario para generar el análisis léxico del programa fuente.

En este archivo también usaremos linkedList para guardar tanto los tokens, como los errores léxicos encontrados por el lexer, para usar luego esas estructuras en los reportes.

```
package olc1.proyecto1.pkg202307793;
import java_cup.runtime.*;
import java.util.LinkedList;
import Clases.token;
import Clases.errorLexico;
%%

//Directrices
%class Lexico
%public
%line
%char
%cup
%unicode
%ignorecase

%{
    LinkedList<errorLexico> listaErrores = new LinkedList<>();
    LinkedList<token> listaTokens = new LinkedList<>();
%}

%init{
    yyline = 1;
    yychar = 1;
%init}
```

En el archivo definimos cada token que usaremos, y lo incluimos de la siguiente forma.

```
// ===== PALABRAS RESERVADAS =====
"Evaluar" {
    token t = new token("REVALUAR", yytext(), (int)yyline, (int)yychar);
    listaTokens.add(t);
    System.out.println("Palabra reservada reconocida: " + yyline + " - " + yychar + " ->> " + yytext());
    return new Symbol(sym.REVALUAR, (int)yyline, (int)yychar, yytext());
},
```

Los errores los gestionamos de manera similar, y los espacios en blanco, los ignoramos.

```
[\t\r\n\f ]      { }

. {
    errorLexico e = new errorLexico(yytext(), (int)yyline, (int)yychar, "Símbolo no reconocido");
    //listaErrores.add(E );
    System.out.println(
        "Error Léxico: " + yytext() +
        " en línea: " + yyline +
        " columna: " + yychar
    );
}
```

2. Librerías usadas - cup - (parser)

La librería que usaremos para el análisis léxico será la librería de CUP. Esta librería necesita un archivo de configuración .cup para que pueda generar el código necesario para generar el análisis sintáctico del programa fuente.

Este archivo contiene declarados los símbolos terminales y no terminales, los terminales, son los mismos que fuimos recogiendo desde el lexer. Los no terminales, son las producciones con el orden establecido de los elementos terminales de nuestro programa fuente.

```
/* ===== TERMINALES ===== */
terminal String PTCOMA, PARIZQ, PARDER, CORIZQ, CORDER, LLAIZQ, LLADER;
terminal String IGUAL, DIFERENTE, MAYOR, MENOR, MAYORIGUAL, MENORIGUAL;
terminal String AND, OR, NOT;
terminal String ENTERO, DECIMAL, ID, STRING_LITERAL;
terminal String TRUE, FALSE;
terminal String INT, FLOAT, BOOL, STRING, ARRAY, OBJECT, NULL;
terminal String DATABASE, USE, TABLE, READ, FIELDS, FILTER, STORE, AT, EXPORT;
terminal String ADD, UPDATE, SET, CLEAR;
terminal String REVALUAR;
terminal String COMA, DOSPUNTOS, IGUAL_SIMPLE;

/* ===== NO TERMINALES ===== */
non terminal programa;
non terminal instruccion;
non terminal listaInstrucciones;

non terminal definicionDB;
non terminal storeDecl;
non terminal instruccionUse;
non terminal definicionTabla;
```

Producciones que utilizamos en nuestro analizador Sintáctico.

```
definicionDB ::=
    DATABASE ID:nombre LLAIQ storeDecl:s LLADER
    {: System.out.println("Definición de base de datos: " + nombre); :}
;

storeDecl ::=
    STORE AT STRING_LITERAL:ruta PTCOMA
    {: System.out.println("Ruta de persistencia: " + ruta); :}
;

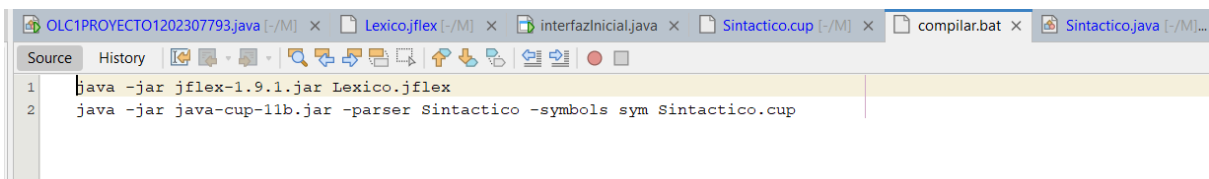
instruccionUse ::=
    USE ID:nombre PTCOMA
    {: System.out.println("Usando base de datos: " + nombre); :}
;

//PARA DEFINIR LAS TABLAS
definicionTabla ::=
    TABLE ID:nombre LLAIQ listaCamposTabla LLADER
    {: System.out.println("Definición de tabla: " + nombre); :}
;

listaCamposTabla ::=
```

3. Archivo de compilación

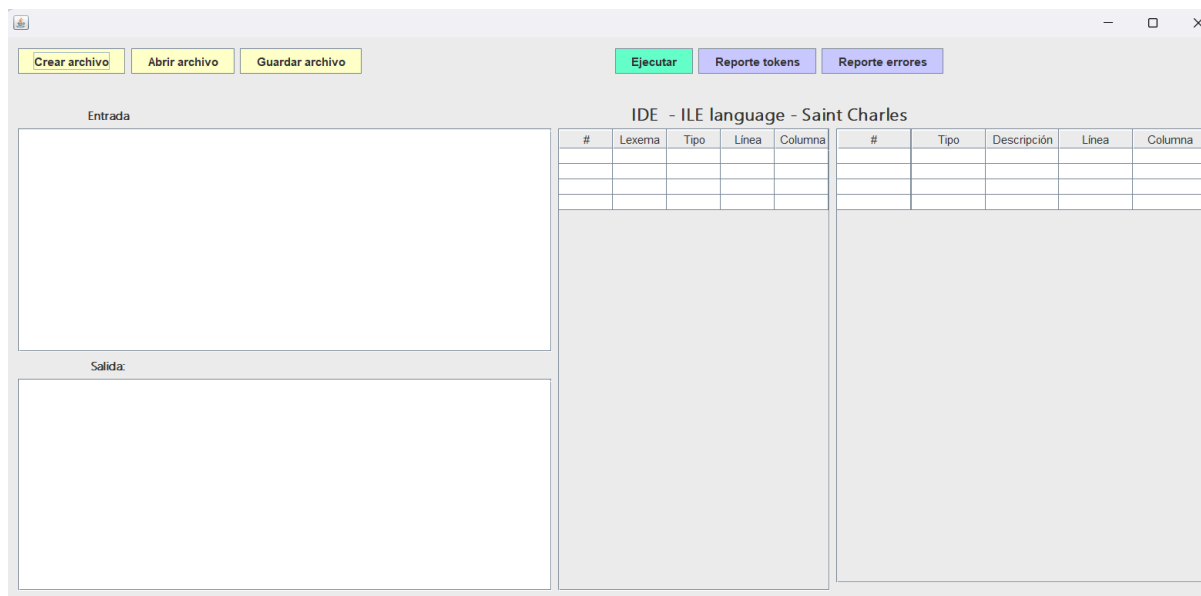
Este archivo ejecuta los archivos de configuración tanto del lexer como el del parser, para así, crear archivos .java funcionales para empezar a analizar el programa fuente que el usuario estaría metiendo dentro del programa. El archivo tiene la siguiente estructura, su terminación es .bat



Interfaz gráfica

4. Vista del IDE

En el archivo de la vista, se colocaron los elementos necesarios para hacer un IDE completamente funcional para leer el lenguaje de programación ELI. Su estructura es la siguiente. Sus componentes son públicos, para una mejor manipulación. Este archivo cuenta con un archivo controlador, para desarrollar toda la lógica del programa.



5. Controlador de la interfaz

En este controlador se desarrollan las funciones correspondientes a la lógica del programa en general.

- Crear archivo nuevo: En esta función se crea una ventana del explorador de archivos, se crea un archivo vacío de terminación .eli. Si el usuario le da al botón de crear archivo en el explorador de archivos, se crea el archivo vacío y se establece como el archivo en el que se está trabajando actualmente.

```

private void crearNuevoArchivo() {
    //creamos un archivo dentro de una carpeta
    //chooser será la entidad que creará el cuadro de explorador de archivos
    JFileChooser chooser = new JFileChooser();
    //este es el título del cuadro del archivo
    chooser.setDialogTitle("Crear nuevo archivo .eli");
    //nombre del nuevo archivo que vamos a crear
    chooser.setSelectedFile(new File("nuevo.eli"));
    //
    if (chooser.showSaveDialog(vista) == JFileChooser.APPROVE_OPTION) {
        //archivo actual será tomado como el archivo que se está ejecutando
        archivoActual = chooser.getSelectedFile();
        try {
            if (archivoActual.createNewFile()) {
                JOptionPane.showMessageDialog(vista, "Archivo creado: " + archivoActual.getName());
            }
            vista.entrada_programa.setText(""); // limpiar área de entrada
        } catch (IOException ex) {
            JOptionPane.showMessageDialog(vista, "Error al crear archivo: " + ex.getMessage());
        }
    }
}

```

- Guardar archivo: En esta función se toma el contenido del área de trabajo del IDE, y procede a escribir el contenido del archivo que se establece como el archivo en el que se está trabajando.

```

private void guardarArchivo() {
    //si hay un archivo creado y en uso, se ejecuta el bloque de dentro
    if (archivoActual != null) {
        //si hay algo que guardar
        try (FileWriter fw = new FileWriter(archivoActual)) {
            fw.write(vista.entrada_programa.getText()); // se toma lo que haya dentro del área de texto
            JOptionPane.showMessageDialog(vista, "Archivo guardado: " + archivoActual.getName());
        } catch (IOException ex) {
            JOptionPane.showMessageDialog(vista, "Error al guardar archivo: " + ex.getMessage());
        }
    } else {
        JOptionPane.showMessageDialog(vista, "Primero crea un archivo nuevo.");
    }
}

```

- Abrir un archivo: En esta función se crea una ventana del explorador de archivos, para que el usuario pueda elegir el archivo .eli que va a ejecutar dentro del IDE.

```

private void abrirArchivo() {
    JFileChooser chooser = new JFileChooser();
    chooser.setDialogTitle("Abrir archivo .eli");
    if (chooser.showOpenDialog(vista) == JFileChooser.APPROVE_OPTION) {
        archivoActual = chooser.getSelectedFile();
        try (BufferedReader br = new BufferedReader(new FileReader(archivoActual))) {
            vista.entrada_programa.read(br, null);
            JOptionPane.showMessageDialog(vista, "Archivo cargado: " + archivoActual.getName());
        } catch (IOException ex) {
            JOptionPane.showMessageDialog(vista, "Error al abrir archivo: " + ex.getMessage());
        }
    }
}

```


Clases para materialización de la BD

6. Clase tabla

Esta clase es utilizada para agregar los registros de la base de datos, además de contener la información del esquema de cada tabla. Las instancias de esta clase serán contenidas dentro de la clase de base de datos.

```
public class Tabla {
    String name;
    //cada fila es un objeto de un hash de fila:valor
    List<Map<String, Object>> rows = new ArrayList<>();
    //lista de columnas, cada columna es una de tipo nombre-tipo
    public List<Map.Entry<String,String>> columnas = new ArrayList<>();

    public Tabla(String name){
        this.name = name;
    }

    //agregar una columna con su tipo
    public void agregarColumna(String nombreCampo, String tipo) {
        columnas.add(new AbstractMap.SimpleEntry<>(nombreCampo, tipo)); }
}
```

7. Clase actualizacionInfo

Esta clase es utilizada para ejecutar los filtros necesarios para las consultas dentro de las tablas de la base datos, ya que contiene información de los campos que vamos a manipular, los operadores que usaremos para ello y los valores que aplicaremos a los filtros.

```
public class actualizacionInfo {
    public List<Map.Entry<String,Object>> campos;
    public String campoFiltro;
    public String operador;
    public Object valorFiltro;
    public actualizacionInfo(List<Map.Entry<String,Object>> campos, String campoFiltro,String operador, Object valorFiltro)
    { this.campos = campos; this.campoFiltro = campoFiltro; this.valorFiltro = valorFiltro; this.operador = operador;
    }
}
```

8. Clase camposInfo

Esta clase es utilizada para guardar los campos que serán leídos o actualizados dentro de las consultas de read o update.

```
*/
public class camposInfo {
    public List<String> campos; //esto sirve para encapsular la sección de campos
}
public camposInfo(List<String> campos) {
    this.campos = campos; }
}
```

9. Clase dataBase

Esta clase es utilizada para guardar las tablas y registros que el usuario del lenguaje eli quiera colocar, la base de datos solamente tiene un nombre, una ruta y el hashmap de los pares <nombre de la tabla, objeto tabla>.

```
public class dataBase {
    String nombre;
    String filePath;
    Map<String, Tabla> tablas = new HashMap<>();
    public dataBase(String name, String filePath) {
        this.nombre = name;
        this.filePath = filePath;
    }
}
```

10. Clase dataBase

Esta clase es utilizada para guardar la información de los errores léxicos encontrados durante la fase del scanner, estos errores serán guardados en una lista para ser impresos una vez termine la ejecución del código del programa.

```
public class errorLexico {
    private String lexema;
    private int linea;
    private int columna;
    private String descripcion;

    public errorLexico(String lexema, int linea, int columna, String descripcion) {
        this.lexema = lexema;
        this.linea = linea;
        this.columna = columna;
        this.descripcion = descripcion;
    }
}
```

11. Clase filtroInfo

Esta clase es utilizada para guardar y para la posterior ejecución de los filtros hacia x tabla, este objeto guarda el campo al que se le aplicará el filtro, además del operador y el valor del filtro.

```
//esto encapsula los filtros
public class filtroInfo {
    public String campo;
    public String operador;
    public Object valor;
    public filtroInfo(String campo, String operador, Object valor) {
        this.campo = campo; this.valor = valor; this.operador = operador;
    }
}
```

12. Clase gestorDB

Esta clase es utilizada como el coordinador de toda la base de datos, acá se maneja la lógica detrás del lenguaje de consultas eli. Solo hay una instancia del gestor cada vez que se ejecuta el software. Los siguientes métodos son los que le dan vida a la base de datos:

```
public class gestorBD {
    //todos los objetos de tipo dataBase que tengamos
    private Map<String, dataBase> bases = new HashMap<>();
    //La base de datos que estamos usando actualmente
    private dataBase baseActual;
    //Objeto Gson para convertir a JSON
    private Gson gson = new Gson();
    //ultimo read visto
    private List<Map<String, Object>> ultimaLectura = new ArrayList<>();
    // Variables de contexto para export, para incluir el esquema y nombre de la tabla
    private String ultimaTablaLeida = "";
    private List<String> ultimosCamposLeidos = new ArrayList<>();
}
```

- última lectura: En este método se devuelve el atributo de última lectura, que es una estructura que guarda el último read ejecutado.

```
public List<Map<String, Object>> getUltimaLectura() {
    return ultimaLectura;
}
```

- Crear base de datos: En este método se toma la ruta de la nueva base de datos, se limpia el nombre para que salga sin comillas extra, se crea un objeto de tipo dataBase, se agrega este objeto a la lista de bases de datos existentes y se marca como base de datos actual(la base de datos que

podremos manipular en este momento) la base recién creada.

```
// Crear base de datos
public void crearBaseDeDatos(String nombre, String ruta) {
    String rutaLimpia = ruta.replace("\\", "");
    DataBase db = new DataBase(nombre, rutaLimpia);
    bases.put(nombre, db); //al crear una nueva bd, la metemos en nuestros registros cawn
    baseActual = db;
    guardar(db);
}
```

- Usar base de datos: En este método, se marca como base de datos actual(la base de datos que podremos manipular en este momento), la base de datos indicada con la instrucción USE.

```
// Usar base de datos existente
public void usarBaseDeDatos(String nombre) {
    baseActual = bases.get(nombre);
}
```

- Crear tabla dentro de una base de datos: En esta instrucción se espera que llegue el nombre de la tabla y un hashmap de tipo <nombre campo, tipo campo>, para la creación de la tabla dentro de la BD, junto con el schema.

```
// Crear tabla
public void crearTabla(String nombreTabla, List<Map.Entry<String,String>> columnas) {
    if (baseActual != null) { //si la bd que usamos actualmente existe, ...
        Tabla t = new Tabla(nombreTabla); //creamos una tabla nueva
        //guardar las columnas en la tabla
        for (Map.Entry<String,String> col : columnas) {
            t.agregarColumna(col.getKey(), col.getValue());
        }
        baseActual.tablas.put(nombreTabla, t); // y la metemos en el atributo hashmap de nuestra bd
        guardar(baseActual); //SIEMPRE, guardamos los cambios
    }
}
```

- Insertar datos: En esta función se espera el nombre de la tabla y un hashmap de tipo <nombre de la tabla, lista de campos y valores a insertar>, pero primero, se tiene que verificar el tipo de dato del campo antes de insertarlo.

```
// Insertar registro
public void insertarRegistro(String nombreTabla, Map<String,Object> fila) {
    Tabla t = baseActual.tablas.get(nombreTabla);
    if (t != null) {
        Map<String,Object> filaLimpia = new LinkedHashMap<>();
        for (Map.Entry<String,Object> entry : fila.entrySet()) {
            String campo = entry.getKey();
            Object valor = entry.getValue();
            // Buscar el tipo del campo en el schema
            String tipo = obtenerTipo(t, campo);
            if (tipo != null) {
                if (tipo.equals("int")) valor = Integer.parseInt(valor.toString());
                else if (tipo.equals("float")) valor = Double.parseDouble(valor.toString());
                else if (valor instanceof String) valor = ((String)valor).replace("\\", "");
            }
            filaLimpia.put(campo, valor);
        }
        t.rows.add(filaLimpia);
        guardar(baseActual);
    }
}

//=====FUNCIÓN AUXILIAR PARA VERIFICAR EL TIPO DE DATO=====
private String obtenerTipo(Tabla t, String campo) {
    for (Map.Entry<String,String> col : t.columnas) {
        if (col.getKey().equals(campo)) return col.getValue();
    }
    return null;
}
```

- Leer : En esta función se recibe el nombre de la tabla a leer, además de recibir el nombre de los campos que se van a leer, el nombre de los campos a los que hay que aplicarles el filtro, el operador del filtro y el valor del propio filtro.

```
// Leer registros con filtro
public List<Map<String,Object>> leer(String nombreTabla, List<String> campos, String campoFiltro, String operador, String valorFiltro) {
    Tabla t = baseActual.tablas.get(nombreTabla); //siempre tomamos la tabla antes de hacerle más de algo
    List<Map<String,Object>> resultado = new ArrayList<>(); //creamos un resultado para insertar en la tabla desj
    if (t != null) {
        for (Map<String,Object> fila : t.rows) { //vamos recorriendo las filas
            if (campoFiltro == null || evaluarFiltro(fila, campoFiltro, operador, valorFiltro)) {
                // Solo incluir los campos seleccionados
                Map<String,Object> filaFiltrada = new LinkedHashMap<>();
                for (String campo : campos) {
                    filaFiltrada.put(campo, fila.get(campo));
                }
                resultado.add(filaFiltrada);
            }
        }
    }
    //mandamos la última información obtenida
}
```

Además usamos una función auxiliar para verificar qué acción hacer con el

filtro.

```
private boolean evaluarFiltro(Map<String,Object> fila, String campo,
                             String operador, Object valor) {
    Object val = fila.get(campo);
    if (val == null) return false;

    try {
        double v1 = Double.parseDouble(val.toString());
        double v2 = Double.parseDouble(valor.toString());
        switch (operador) {
            case "==" : return v1 == v2;
            case "!=" : return v1 != v2;
            case ">"  : return v1 > v2;
            case "<"  : return v1 < v2;
            case ">=" : return v1 >= v2;
            case "<=" : return v1 <= v2;
        }
    } catch (NumberFormatException e) {
        // Comparación de strings
        String s1 = val.toString().replace("\\", "");
        String s2 = valor.toString().replace("\\", "");
        switch (operador) {
            case "==" : return s1.equals(s2);
            case "!=" : return !s1.equals(s2);
        }
    }
}
```

- Actualizar registros: En esta función se recibe el nombre de la tabla a actualizar, además de recibir un hash de los campos que se van a actualizar con <nombre, valor>, el nombre del campo al que le aplicaremos el filtro, y el valor del propio filtro.

```
// Actualizar registros
public List<Map<String,Object>> actualizar(String nombreTabla, List<Map.Entry<String,
Tabla t = baseActual.tablas.get(nombreTabla);
List<Map<String,Object>> resultado = new ArrayList<>();
if (t != null) {
    for (Map<String,Object> fila : t.rows) {
        if (campoFiltro == null || fila.get(campoFiltro).equals(valorFiltro)) { //var
            // aplicar actualizaciones
            for (Map.Entry<String,Object> entry : campos) {
                fila.put(entry.getKey(), entry.getValue());
            }
            resultado.add(fila);
        }
    }
    guardar(baseActual); //SIEMPRE GUARDAMOS LOS CAMBIOS
}
return resultado;
```

- Limpiar la tabla: Esta función es fácil de leer, solamente se eliminarán todas las líneas que tiene una tabla X, y después se guardará la base de datos.

```
// Limpiar tabla
public void limpiarTabla(String nombreTabla) {
    Tabla t = baseActual.tablas.get(nombreTabla);
    if (t != null) {
        t.rows.clear(); //Eliminamos todas las líneas de la tabla
        guardar(baseActual); //SIEMPRE, guardamos los cambios
    }
}
```

- Exportar una base de datos: Se toma el nombre del archivo, luego se limpia, se busca la última tabla leída, junto con los últimos campos leídos, se crea un objeto llamado export, se comprueban los datos de cada campo para la parte del schema, se ingresa el nombre de la tabla, se cargan los registros en los récords, y se meten dentro del objeto export. Por último se exporta el archivo JSon.

```
//Exportar sin datos
public void exportar(String nombreArchivo) {
    String nombreLimpio = nombreArchivo.replace("\\", "");
    Tabla t = baseActual.tablas.get(ultimaTablaLeida);
    if (t == null) {
        System.err.println("Error: no hay lectura previa para exportar");
        return;
    }

    // Construir objeto de exportación con la estructura correcta
    Map<String, Object> exportObj = new LinkedHashMap<>();
    exportObj.put("table", ultimaTablaLeida);

    // campos con sus tipos desde el esquema de la tabla
    Map<String, String> fields = new LinkedHashMap<>();
    for (String campo : ultimosCamposLeidos) {
        String tipo = obtenerTipo(t, campo);
        fields.put(campo, tipo != null ? tipo : "string");
    }
    exportObj.put("fields", fields);

    //ahora toca meter lo de los records
```

- Guardar una base de datos: Es un proceso parecido al de exportar, solamente que acá creamos un objeto de tipo bd, donde meteremos el nombre de la bd, las tablas con sus schemas, y records. La base de datos se guarda de manera general, entonces se van recorriendo todas las tablas de la base de datos que se pase a este método. Por último pasamos este mega objeto de nombre bd

a archivo Json.

```
// Guardar base de datos completa en JSON, siempre ir refrescando
private void guardar(dataBase db) {
    String nombreLimpio = db.filePath.replace("\\", "");
    //para darle forma al json, se debe de crear un objeto db, contendrá todo lo demás
    Map<String, Object> dbObj = new LinkedHashMap<>();
    dbObj.put("database", db.nombre);
    //luego siguen las tablas, esta es una lista de tablas
    Map<String, Object> tables = new LinkedHashMap<>();
    //hay que meterle info a las tablas
    for (Map.Entry<String, Tabla> entry : db.tablas.entrySet()) {
        //t es la variable iteradora
        Tabla t = entry.getValue();

        // schema con los tipos
        Map<String, String> schema = new LinkedHashMap<>();
        for (Map.Entry<String, String> col : t.columnas) {
            schema.put(col.getKey(), col.getValue()); //para sacar la info de las columnas
            //sería algo tipo 'campo' y su valor , pues es su tipo
        }
    }
}
```

- Cargar una base de datos: Primero vamos a buscar todos los archivos de tipo json dentro de la raíz del proyecto, y los vamos a recuperar.

```
//para poder meter todas la estructuras necesarias
public void cargarTodasLasBases() {
    java.io.File directorio = new java.io.File(".");
    java.io.File[] archivos = directorio.listFiles(
        (dir, name) -> name.endsWith(".json") // solo buscaremos json
    );
    if (archivos != null) {
        for (java.io.File archivo : archivos) {
            cargarBaseDeDatos(archivo.getName());
        }
    }
}
```

Luego de ir cargando los archivos para la base de datos, primero, se verifica que en la raíz tenga la palabra 'dataBases' o 'tables', si no, tira error. Con ayuda de la librería Gson, se aplica la inversa a los archivos json, para sacar de los json, la información necesaria para volver a crear objetos de nuestro

programa, como dataBase, Tabla, y los registros correspondientes.

```
//creamos el objeto de la bd
String nombreDB = root.get("database").getAsString();

//ahora vamos a crear las tablas
dataBase db = new dataBase(nombreDB, nombreLimpio);
//meter la db a nuestra lista de db
bases.put(nombreDB, db);

//vamos a reconstruir las tablas, tomando los tables
com.google.gson.JsonObject tables = root.getAsJsonObject("tables");
for (Map.Entry<String, com.google.gson.JsonElement> tablaEntry : tables.entrySet()) {
    String nombreTabla = tablaEntry.getKey();
    Tabla t = new Tabla(nombreTabla);

    com.google.gson.JsonObject tablaObj = tablaEntry.getValue().getAsJsonObject();

    // Reconstruir el schema
    com.google.gson.JsonObject schema = tablaObj.getAsJsonObject("schema");
    for (Map.Entry<String, com.google.gson.JsonElement> col : schema.entrySet()) {
        t.agregarColumna(col.getKey(), col.getValue().getAsString()); //agregamos la columna del objeto sc
    }
}
```

13. Clase lecturaInfo

Esta clase es utilizada para ejecutar los filtros necesarios para las consultas de read dentro de las tablas de la base datos, ya que contiene información de los campos que vamos a manipular, los operadores que usaremos para ello y los valores que aplicaremos a los filtros.

```
/**
 *
 */
public class lecturaInfo {
    public List<String> campos; //campos seleccionados
    public String campoFiltro; //campo usado en el filtro
    public String operador;
    public Object valorFiltro; //valor del filtro
    public lecturaInfo(List<String> campos, String campoFiltro, String operador, Object valorFiltro) {
        this.campos = campos;
        this.campoFiltro = campoFiltro;
        this.operador = operador;
        this.valorFiltro = valorFiltro; }
}
```

14. Clase token

Esta clase es la base de este proyecto, ya que usamos la clase token para poder llevar un control sobre todas las entradas que tenemos desde el analizador léxico. Estas instancias se quedan guardadas en una linked list dentro de. controlador.

```
/**
 *
 */
public class token {

    private String tipo;
    private String lexema;
    private int linea;
    private int columna;

    public token(String tipo, String lexema, int linea, int columna) {
        this.tipo = tipo;
        this.lexema = lexema;
        this.linea = linea;
        this.columna = columna;
    }
}
```

Orden de las carpetas del proyecto

Este es el orden de las carpetas que uso en este proyecto, además de los archivos y librerías que también se usaron para la realización del mismo:

- ▼  OLC1-PROYECTO1-202307793 [main]
 - ▼  Source Packages
 - ▼  Clases
 -  Tabla.java
 -  actualizacionInfo.java
 -  camposInfo.java
 -  dataBase.java
 -  errorLexico.java
 -  filtroInfo.java
 -  gestorBD.java
 -  lecturalInfo.java
 -  token.java
 - ▼  Interfaz
 -  interfazInicial.java
 - ▼  controladores
 -  controladorInterfaz.java
 - >  olc1.proyecto1 [main]
 - ▼  olc1.proyecto1.pkg202307793
 -  Lexico.java [-/M]
 -  Lexico.jflex [-/M]
 -  OLC1PROYECTO1202307793.java [-/M]
 -  Sintactico.cup [-/M]
 -  Sintactico.java [-/M]
 -  compilar.bat
 - >  gson-2.13.2.jar [-/A]
 - >  java-cup-11b-runtime.jar
 - >  java-cup-11b.jar
 - >  jflex-1.9.1.jar
 -  sym.java [-/M]
 - >  Test Packages
 - ▼  Libraries
 - >  java-cup-11b-runtime.jar
 - >  java-cup-11b.jar
 - >  jflex-1.9.1.jar
 - >  gson-2.13.2.jar
 - >  JDK 25 (Default)
 -  Test Libraries